

Асинхронний thread-safe CRUD сервіс для веломагазину

Початок паралельного створення велосипедів...
 Очікування сигналу від AutoResetEvent...
 Сигнал отримано. Частина об'єктів уже створена.

Демонстрація SemaphoreSlim:

Потік 0 отримав доступ до обмеженого ресурсу
 Потік 2 отримав доступ до обмеженого ресурсу
 Потік 1 отримав доступ до обмеженого ресурсу
 Потік 3 отримав доступ до обмеженого ресурсу
 Потік 4 отримав доступ до обмеженого ресурсу
 Потік 5 отримав доступ до обмеженого ресурсу
 Потік 6 отримав доступ до обмеженого ресурсу
 Потік 8 отримав доступ до обмеженого ресурсу
 Потік 9 отримав доступ до обмеженого ресурсу
 Потік 7 отримав доступ до обмеженого ресурсу

Усі додаткові потоки завершили роботу.

Кількість створених велосипедів: 1000

Статистика по цінах:

Мінімальна ціна: 8097 грн
 Максимальна ціна: 59972 грн
 Середня ціна: 34671,96 грн

Статистика по розміру коліс:

Мінімальний розмір коліс: 26
 Максимальний розмір коліс: 29

Перша сторінка, 10 велосипедів:

Велосипед: Urban Ride, бренд: Scott, рама: S, колеса: 28, ціна: 50853 грн
 Велосипед: Road Speed, бренд: Scott, рама: S, колеса: 28, ціна: 13058 грн
 Велосипед: City Comfort, бренд: Cube, рама: S, колеса: 26, ціна: 12684 грн
 Велосипед: Urban Ride, бренд: Trek, рама: M, колеса: 26, ціна: 34615 грн
 Велосипед: Urban Ride, бренд: Cube, рама: L, колеса: 26, ціна: 56088 грн
 Велосипед: Gravel Expert, бренд: Merida, рама: S, колеса: 29, ціна: 19810 грн
 Велосипед: Road Speed, бренд: Trek, рама: L, колеса: 29, ціна: 27715 грн
 Велосипед: Road Speed, бренд: Cube, рама: S, колеса: 27, ціна: 13441 грн
 Велосипед: City Comfort, бренд: Scott, рама: S, колеса: 26, ціна: 20168 грн
 Велосипед: City Comfort, бренд: Scott, рама: L, колеса: 26, ціна: 29119 грн

Перевірка роботи IEnumerable, перші 5 елементів:

Велосипед: Urban Ride, бренд: Scott, рама: S, колеса: 28, ціна: 50853 грн
 Велосипед: Road Speed, бренд: Scott, рама: S, колеса: 28, ціна: 13058 грн
 Велосипед: City Comfort, бренд: Cube, рама: S, колеса: 26, ціна: 12684 грн
 Велосипед: Urban Ride, бренд: Trek, рама: M, колеса: 26, ціна: 34615 грн
 Велосипед: Urban Ride, бренд: Cube, рама: L, колеса: 26, ціна: 56088 грн

Колекцію збережено у файл: async_bikes.json

C:\Users\User\Desktop\lab2\BikeShop.Console\bin\Debug\net10.0\BikeShop.Console.exe (process 14020) exited with code 0 (0x0).

Press any key to close this window . . .|

3. Що таке асинхронність, яка різниця між асинхронністю та багатопотоковістю?

Асинхронність — це спосіб виконання операцій (наприклад, запиту до бази чи мережі) без зупинки основного потоку: ми відправляємо запит і звільняємо потік для інших завдань, поки чекаємо на відповідь. Головна різниця в тому, що багатопотоковість — це «робити кілька речей одночасно» (використовуючи різні потоки), а асинхронність — це «не чекати на завершення операції, а займатися іншим» (це може відбуватися навіть в одному потоці).

4. Для чого використовуються ключові слова `async/await`?

`async` позначає метод як такий, що містить асинхронні операції, а `await` вказує компілятору призупинити виконання методу до завершення очікуваної задачі, не блокуючи при цьому потік виконання. Це дозволяє писати асинхронний код, який виглядає та читається як послідовний.

5. Чим відрізняється `Task` від `ValueTask`?

`Task` — це посилальний тип (reference type), який завжди створюється в купі (heap), що створює навантаження на Garbage Collector. `ValueTask` — це структура (value type), яка може бути ефективнішою, якщо операція завершується синхронно або результат уже готовий, оскільки вона дозволяє уникнути зайвого виділення пам'яті.

6. Що таке thread-safe колекція? Які приклади таких колекцій у .NET ви знаєте?

Це колекції, розроблені для безпечної роботи з кількох потоків одночасно без ризику пошкодження даних. Приклади з простору `System.Collections.Concurrent`: `ConcurrentDictionary<K, V>`, `ConcurrentQueue<T>`, `ConcurrentStack<T>`, `ConcurrentBag<T>`.

7. Для чого використовуються примітиви синхронізації, такі як `lock`, `Semaphore`, `AutoResetEvent`?

Вони потрібні для керування доступом до спільного ресурсу, щоб уникнути «стану гонитви» (race condition). `lock` гарантує, що лише один потік увійде в критичну секцію. `Semaphore` обмежує кількість потоків, що мають доступ одночасно. `AutoResetEvent` використовується для сигналізації між потоками (один чекає на подію від іншого).

8. Як забезпечити безпеку при одночасному зверненні кількох потоків до спільного ресурсу?

Найкращий спосіб — використання блокувань (`lock`) для критичних ділянок коду, використання thread-safe колекцій або розробка коду так, щоб потоки не ділили між собою змінні стану (незмінні/immutable об'єкти).

9. Як за допомогою LINQ можна отримати мінімальне, максимальне та середнє значення певної властивості?

Використовуються методи розширення `Min()`, `Max()` та `Average()`: `var min = list.Min(x => x.Value); var max = list.Max(x => x.Value); var avg = list.Average(x => x.Value);`

10. У чому різниця між методами `Select`, `Where`, `Aggregate`, `OrderBy`?

- **Select:** трансформує кожен елемент у новий вигляд (проекція).
- **Where:** фільтрує елементи за умовою.
- **Aggregate:** згортає послідовність у одне значення через накопичення (наприклад, сума).
- **OrderBy:** сортує елементи послідовності.

11. Які переваги використання LINQ у порівнянні з класичними циклами?

LINQ робить код більш декларативним, лаконічним, легшим для читання та підтримки. Він зменшує ймовірність помилок, пов'язаних з керуванням лічильниками циклів, і дозволяє легко комбінувати операції над даними.

12. Що станеться, якщо два потоки одночасно спробують зберегти колекцію у файл?

Виникне помилка доступу до файлу (виключення `IOException`), оскільки операційна система блокує файл для запису одним процесом або потоком. Дані можуть бути пошкоджені або файл просто залишиться заблокованим.

13. Як працює `Parallel.For` і у яких випадках його краще використовувати?

`Parallel.For` розбиває ітерації циклу на кілька потоків, що дозволяє виконувати їх паралельно. Це доцільно використовувати для «важких» обчислювальних задач, які не залежать одне від одного, на багатоядерних процесорах. Не варто використовувати для простих операцій або роботи з небезпечними ресурсами без синхронізації.

14. Що таке пагінація і як вона реалізована у вашій системі?

Пагінація — це поділ великого обсягу даних на окремі сторінки для відображення по частинах. У системах це зазвичай реалізується через параметри запиту до бази даних: `Skip(n)` (пропустити n записів) та `Take(m)` (взяти m записів на сторінку).